

Finding Optimal Cyclic Quorum Systems

Combinatorial Search and Deep Reinforcement Learning

Wai-Shing Luk

Fudan University

September 25, 2026

- 1 Introduction
- 2 An Example
- 3 Finding Difference Covers
- 4 Applications
- 5 Results
- 6 Implementation and Performance
- 7 Conclusions

Section 1

Introduction

The Coordination Problem

- Distributed algorithms must coordinate many independent sites
- In mutual exclusion, only one site may enter the critical section at a time
- We want **equal work**, **equal responsibility**, and a small quorum size to limit communication
- Finding optimal systems for arbitrary N is a hard combinatorial problem

Quorum Systems

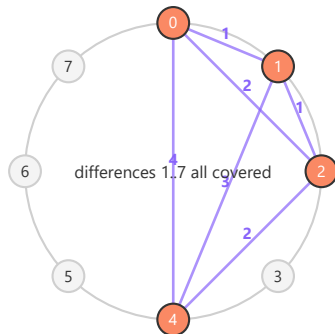
- A quorum system is a family $\mathcal{Q}$ of subsets of the sites, called quorums
- **Intersection property:** $G \cap H \neq \emptyset$ for all $G, H \in \mathcal{Q}$
- Any two operations therefore meet at a common site, which permits conflict detection and resolution
- Maekawa: a symmetric system on N sites needs quorum size d with $d(d-1) + 1 \geq N$, so $d \geq \lceil \sqrt{N} \rceil$
- Grid-based systems are simpler, but use quorums of size about $2\sqrt{N} - 1$

Cyclic Quorum Systems

- Fix a **base quorum** $B_0 = \{a_1, \dots, a_d\} \subseteq \{0, \dots, N-1\}$
- Generate every quorum by a cyclic shift: $B_i = \{a_1 + i, \dots, a_d + i\} \pmod{N}$
- Equal work and equal responsibility then hold automatically
- A CQS is thus described by a single base quorum

Difference Sets and Covers

- A set $\{a_1, \dots, a_d\}$ is a cyclic (N, d, λ) -**difference set** if every non-zero residue occurs exactly λ times as a difference
- A **relaxed** (N, d) -difference set requires every non-zero residue to occur at least once
- A base quorum yields a valid CQS exactly when it is a relaxed difference set
- A CQS also satisfies **rotation closure**:
 $G \cap (H + i) \neq \emptyset$ for all quorums G, H and every shift i
- Singer difference sets exist when
 $N = d(d - 1) + 1$ with $d - 1$ a prime power



Section 2

An Example

Building a CQS for $N = 8$

- Base quorum $B_0 = \{0, 1, 2, 4\}$ (mod 8), of size $d = 4$
- Shifting cyclically:
 - $B_1 = \{1, 2, 3, 5\}$
 - $B_2 = \{2, 3, 4, 6\}$
 - $B_3 = \{3, 4, 5, 7\}$
 - $B_4 = \{4, 5, 6, 0\}, \dots$
- Each site occurs in four quorums, and any two quorums intersect

	0	1	2	3	4	5	6	7
B0	0	1	2		4			
B1		1	2	3		5		
B2			2	3	4		6	
B3				3	4	5		7
B4	0				4	5	6	
B5		1				5	6	7
B6	0		2				6	7
B7	0	1		3				7

Section 3

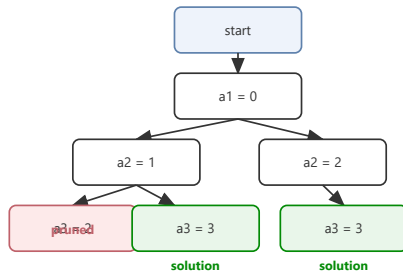
Finding Difference Covers

The Difference Cover Problem

- Choose d markers on a circle of N points
- The pairwise distances must cover $1, \dots, N - 1$ at least once
- Equivalent to a relaxed (N, d) -difference set with minimal d
- Assumptions: $N, d \geq 3$ and $N \leq d(d - 1) + 1$

Approach 1: Recursive Search

- Build the set one element at a time (generate and test)
- Track which differences have already been covered
- **Prune** as soon as the remaining choices cannot cover all differences
- **Backtrack** at a dead end
- Guaranteed to find a solution within its search space

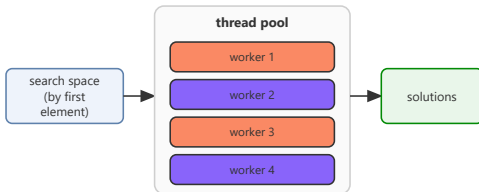


Symmetry Breaking

- Many base quorums are equivalent under rotation or reflection
- Enumerate them through **fixed-density necklaces** (rotation) and **bracelets** (rotation and reflection)
- Generation runs in **constant amortized time (CAT)**: total work is proportional to the number of outputs
- The bracelet algorithm handles arbitrary alphabet size, listing each bracelet once in lexicographic order
- Necklaces can be counted with **Burnside's lemma**; related objects are Lyndon words and De Bruijn sequences

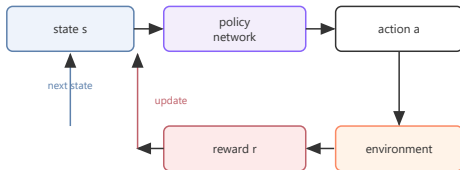
Parallel Search

- Split the search by the value of the first element
- Each partition runs in its own thread from a thread pool
- All cores work simultaneously, greatly reducing wall-clock time
- Each candidate is validated by counting the distinct differences



Approach 2: Reinforcement Learning

- Treat the search as a game: pick the numbers one at a time
- **State:** which numbers are chosen and which differences are covered ($2N$ values)
- **Action:** pick the next number; **reward:** how many new differences it covers
- A policy network (three layers, ReLU, softmax) learns the strategy
- Not guaranteed to find a solution within a fixed number of episodes



The Policy Network

- Input: the $2N$ -dimensional state vector
- Hidden layers use ReLU; the output gives logits over the N numbers
- Softmax turns the logits into a distribution; already chosen numbers are masked
- Parameters are updated by a policy-gradient rule:

$$\text{parameter} \leftarrow \text{parameter} - \eta \times \text{gradient}$$

Parallel Reinforcement Learning

- Several worker threads, each acting as an independent agent
- They **share one policy network** and contribute their gradients
- Mutexes protect the shared network and the output stream
- Learning stops once a solution is found or MAX_EPISODES is reached

Comparison

- **Recursive search:** systematic, with explicit pruning; guaranteed within its search space
- **Reinforcement learning:** learned and reward-driven; not guaranteed
- Both exploit parallelism for speed
- RL learns to find *a* solution, not all of them

Section 4

Applications

Distributed Mutual Exclusion

- A site requests access from every member of its quorum
- The members check for conflicts; the site enters the critical section only when they agree
- The quorum size approaches $\sqrt{N}$
- The cyclic construction gives symmetry from a single base quorum

Distributed All-Pairs Algorithms

- Each process stores only a quorum of the data
- **All-pairs property:** every pair of data sets occurs together in some quorum
- The quorum size grows as $O(\sqrt{P})$ in the number of processes P
- Reported: up to 7x speedup on 8 nodes with a 2/3 memory reduction

Wireless Sensor Networks

- Nodes have different power-saving requirements
- **CQS-Pair** combines two cyclic quorum systems with different cycle lengths
- It guarantees discovery within every m consecutive slots
- This balances energy consumption against discovery delay

Deep Learning and String Algorithms

- **CQS-Attention** scales self-attention to very long sequences
- It reduces the memory bottleneck of the standard computation
- **Suffix-array construction:** the DC3 algorithm samples positions by residue modulo 3
- It relies on the difference-cover property that any two positions share a small offset at which both are sampled

Section 5

Results

Extending the Search

- Earlier work tabulates base quorums for $N = 4$ to 111
- The recursive search extends the optimal base quorums to $N = 150$
- The resulting quorum size stays close to $\sqrt{N}$
- For larger N , the RL agent finds **nearly optimal** base quorums

Optimal and Nearly Optimal Base Quorums

- Optimal base quorums for $N = 112$ to 150 (recursive search)
- Nearly optimal base quorums for $N = 151$ to 171 (RL)
- The complete tables are given in the paper

Section 6

Implementation and Performance

Symmetry Pruning

- The original search broke only rotation symmetry (necklaces)
- Adding the reflection test `CheckRev` reduces it to bracelets
- Reflection only helps near the symmetric frontier, so the gain is modest
- Measured speedup **1.07-1.17x**, with output identical to the reference solver (12 cases)

The RL Agent That Never Learned

- One implementation computed the gradient into a local vector and then **discarded it**
- The update subtracted zero from every weight: a fixed random policy in disguise
- Mean residue coverage stayed flat (about 12.3); a genuine gradient raises it to about 13.9
- A second bug had the **entropy-bonus sign** inverted, collapsing the policy (5/16 solved; fixed: 11/16)

RL Efficiency

- Cache activations during the episode instead of re-running the forward pass
- Reuse per-thread buffers across episodes
- Together these give **1.48x** on a single thread
- Removing the update mutex (Hogwild async SGD) gives **9.5x** on 10 threads
- The resulting data race is deliberate and documented

Robustness

- The guard $N \leq d(d-1) + 1$ overflowed in 32-bit arithmetic; compute the product in 64-bit
- Fuzz the command line with degenerate and overflow inputs: all rejected cleanly
- Rebuild with bounds assertions enabled: no out-of-range access

Performance Summary

Change	Effect
Bracelet symmetry	1.07-1.17x, identical output
Real backpropagation	coverage 12.3 to 13.9
Entropy sign fix	11/16 vs 5/16 solved
Cache and buffer reuse	1.48x single-thread
Lock-free update	9.5x on 10 threads
Overflow-safe guard	no crashes

- Two of these were **bugs**, not tuning

Section 7

Conclusions

Conclusions

- Optimal CQS reduce to finding relaxed (N, d) -difference sets
- A systematic recursive search is guaranteed and enumerates bracelets in CAT
- A parallel RL agent finds nearly optimal base quorums for larger N
- Both approaches exploit parallelism to handle the combinatorial search

Future Work

- Extend the search to larger N
- Improve the RL agent with reward shaping and larger networks
- Generate k -ary necklaces with fixed content for $k > 2$
- List other restricted classes of bracelets